ON THIS PAGE

SCTP CE12 · GROUP 5 CAPSTONE · RETAIL-STORE-GRP5

The Platform, Explained in Plain English

Everything this repository builds and why — written so you can read it once and then explain any part of it to someone else without notes. No prior Kubernetes or AWS knowledge assumed.

Cluster: **retail-store-grp5** Region: **ap-southeast-1**

Order of operations: **Terraform → Helmfile → Kustomize**

START HERE

The one-minute version

Five people built a working online store on AWS, using the same category of tools real engineering teams use to run production software: infrastructure written as code, containers orchestrated by Kubernetes, deployments driven by Git instead of by hand, and a full stack for watching what's happening and knowing when it breaks.

The store itself — a catalogue, a cart, checkout, orders, and a UI that ties them together — isn't the point. It's a stand-in. The five container images are pulled pre-built from AWS's own reference app; nobody on the team wrote or owns that application code. The actual capstone is everything *around* the app: how do you provision a cluster the same way every time, ship a change without anyone typing a deploy command, get paged the moment something breaks, and know what it's costing you to run.

That's what the rest of this page walks through, roughly in the order you'd explain it to someone who's never seen the repo.

THE CAST

What each tool actually does

Ten names get thrown around a lot in this repo. Here's what each one is actually for, in one sentence.

TOOL	JOB TITLE	WHAT IT DOES, IN PLAIN TERMS
Terraform	Infrastructure builder	Writes down, as code, exactly which AWS resources should exist. Run one command and it builds — or rebuilds — the whole network and cluster the same way every time, instead of someone clicking through the AWS console.
Amazon EKS	Container runtime	AWS's managed Kubernetes. Decides which physical server each piece of the app runs on, restarts anything that crashes, and moves work off a server that dies.
Helmfile	Platform installer	A shopping list of "Helm charts" — pre-packaged installs of software like the load balancer controller or Prometheus — applied in the right order.
Kustomize	App deployer	Applies the plain YAML files describing the store's five services. No templating language, just layered configuration.
ArgoCD	Auto-pilot	Watches this GitHub repo and applies any change automatically, usually within about three minutes — nobody has to run a deploy command.
Prometheus + Grafana	Metrics & dashboards	Prometheus counts things (CPU, memory, requests, errors) every few seconds; Grafana draws those numbers as the dashboards the team looks at.
Fluent Bit + Loki	Log pipeline	Fluent Bit tails every container's log output and ships it off; Loki stores it and makes it searchable from inside Grafana.
OpenTelemetry + X-Ray	Request tracing	Follows one shopping request as it hops between services, so you can see exactly which one was slow.

TOOL	JOB TITLE	WHAT IT DOES, IN PLAIN TERMS
Alertmanager → Discord	The messenger	When Prometheus notices something's actually wrong, Alertmanager pushes a message straight into the team's Discord.
Kubecost	Cost tracking	Breaks the AWS bill down by Kubernetes namespace, so "catalog costs $X a month" has an actual answer instead of one lump invoice.
external-dns	DNS robot	Watches the cluster for new routes and automatically writes the matching Route 53 DNS record — nobody edits DNS by hand.
AWS FIS	Professional troublemaker	Deliberately kills real EC2 servers on command, to prove the alerts and self-healing actually work instead of just trusting the YAML.

DEPLOYMENT ORDER

The three layers, in order

Everything in this repo goes down in exactly one sequence, and each layer depends on the one before it existing first. Think of it like building a house: you can't install appliances before the wiring exists, and nobody can move in before the appliances work.

LAYER 1 OF 3 — TERRAFORM/

Terraform — the land and the utilities

Builds the empty cluster and everything it needs to exist on AWS. Runs once, changes rarely.

- **A private network (VPC)** the cluster lives inside, split across three availability zones with one shared NAT gateway for outbound internet access.
- **The EKS cluster itself**, split into two separate groups of servers ("node groups") that never mix:
  - `application` — 3 to 5 general-purpose servers, open to any workload, running the actual store.
  - `observability` — 2 to 3 servers, pinned to a single AWS zone (`ap-southeast-1c`) so their attached disks never get stranded in the wrong zone, and "tainted" so *only* monitoring software is allowed to land there. A taint

works like a bouncer: nothing gets scheduled onto that server unless it explicitly says it's on the list.

- **Permissions, without passwords.** Instead of handing a program an AWS access key that can leak, each Kubernetes "service account" (the load balancer controller, Fluent Bit, Grafana, Loki, the tracing collector, the chaos-testing role) is given a scoped-down IAM role it can borrow temporarily, tied to its own identity. This pattern is called IRSA.
- **A shared memory.** Terraform needs to remember what it has already built. That memory (its "state") lives in one S3 bucket that all five team members' laptops read and write, with built-in locking so two people can't apply changes at the same time and corrupt it.

↓ once the cluster and network exist

LAYER 2 OF 3 HELM/

Helmfile — the appliances

Installs the platform software the cluster needs before the store can run on top of it. Changes occasionally.

A "Helm chart" is a pre-packaged, configurable install of a piece of software — think of it as an app-store listing, but for Kubernetes. Helmfile is the shopping list that installs several charts in a specific order, since some depend on others:

1. **AWS Load Balancer Controller** — before it installs, a hook applies the Gateway API "CRDs" (see below), since the controller's own resources depend on them existing first.
2. **external-dns** — waits for the load balancer controller, since it needs to see load balancers to point DNS at.
3. **Prometheus stack** (kube-prometheus-stack) — two hooks run first: one binds the reusable, pre-created AWS disks (EBS volumes) for Prometheus/Grafana/Loki/Alertmanager/Kubecost so their data survives a teardown, the other installs Prometheus's own CRDs.
4. **Loki** — the log storage backend.
5. **OpenTelemetry Operator** — the piece that later injects tracing into app pods automatically.

6. **Kubecost** — waits for Prometheus, since it reads its metrics to calculate cost.
7. **ArgoCD** — installed last, ready to take over once the app layer below is applied for the first time.

> **What's a CRD?** A Custom Resource Definition teaches Kubernetes a brand-new type of object it didn't understand before — like adding a new shape of LEGO brick to the box. You can't build anything with a brick that hasn't been added yet, which is exactly why this install order matters: skip ahead, and the next layer fails with a "no matches for kind" error.

↓ once the platform software is running

LAYER 3 OF 3 — MANIFESTS/

Kustomize — moving in

Deploys the actual retail store and everything app-specific. Changes constantly — this is what the team iterates on day to day.

- The five store services — `carts`, `catalog`, `checkout`, `orders`, `ui` — each in their own namespace, each following the same file pattern: a namespace, a service account, a config map, a deployment, a service, and (if it needs one) a paired database deployment.
- Supporting pieces: Fluent Bit (log shipping), the tracing collector, a small cron job that generates fake shopper traffic so dashboards have something to show, the alert rules, and the routing that exposes Grafana/Kubecost/ArgoCD to the outside world.
- The ArgoCD `Application` resource itself — this is the one thing in this layer that changes everything downstream. The moment it's applied, ArgoCD starts watching this repo and takes over deploying every future change automatically (see How a change ships).

REFERENCE

File-by-file map — what links to what

The layers above explain the shape of the system. This is the wiring diagram underneath it: for each significant file, what it creates and exactly which other file downstream depends on that — by name, not just by category. Click any file to expand it. Skim this once and you'll never have to grep the repo to answer "wait, where does this Secret/role/URL actually come from?" again.

terraform/

- **`vpc.tf`** — the private network (VPC), 3 AZs, one shared NAT gateway
- **`eks.tf`** — the EKS cluster and its two node groups
- **`iam.tf`** — one narrowly-scoped IAM role per service account (IRSA), plus the FIS role
- **`variables.tf`** — `cluster_admins` list and `loki_bucket_name`

providers.tf's S3 backend (bucket `capstone-project-group5`) is Terraform's own state file. Don't confuse it with the Loki chunks bucket above — two different buckets, two different jobs, similarly-named project.

helm/

- **`helmfile.yaml.gotmpl`** — the 7 releases and the order they install in
- **`crds/kustomization.yaml`** — pulled by the LB controller's presync hook, before anything else installs
- **`retained-storage.yaml`** — the 5 PVs that make dashboard/log data survive a teardown
- **`values/prometheus-stack.yaml`** — the busiest values file in the repo
- **`values/loki.yaml`** — log storage backend
- **`values/opentelemetry-operator.yaml`** — the tracing operator

- **`values/kubecost.yaml`** — cost visibility, reading Prometheus's own data
- **`values/argocd.yaml`** — GitOps controller for the app layer

`manifests/`

Root `manifests/kustomization.yaml` resource order: `catalog, carts, checkout, orders, ui, fluentbit, adot, otel-instrumentation, grafana, load-gen, kubecost, alerts, argocd`. The last five of those target namespace `monitoring` — which no manifests/ file creates. It's created by the Helmfile `prometheus` release's presync hook. Run Kustomize before Helmfile has fully synced and those five fail outright on a missing namespace/CRD.

THE FIVE STORE SERVICES

- **`carts/`**
  — basket contents, backed by an in-cluster DynamoDB-Local fake, not real AWS DynamoDB
- **`catalog/`** — product list, MySQL-backed, the only Go service
- **`checkout/`** — cart-to-order conversion, Redis-backed
- **`orders/`** — completed orders, PostgreSQL-backed
- **`ui/`** — the storefront, and the one Gateway every other route hangs off

SUPPORTING THE PLATFORM

- **`fluentbit/fluent-bit.yaml`** — log shipping DaemonSet
- **`adot/`** — the OpenTelemetryCollector that receives every service's traces
- **`otel-instrumentation/instrumentation.yaml`**
  — the CR every app's inject-* annotation points at

› **`grafana/kustomization.yaml`** — bundles the dashboard JSON into ConfigMaps

› **`load-gen/cronjob.yaml`** — synthetic shopper traffic, every minute

› **`alerts/`** — the two PrometheusRules and the Discord AlertmanagerConfig

› **`argocd/application.yaml`** — the self-managing bootstrap Application

Two things that are load-bearing but have zero file in this repo: the `discord-webhook` Secret (Alertmanager → Discord) and the `repo-ce12-capstone-sandbox` Secret (ArgoCD's read-only git credential). Both are created once, by hand, per the README's setup steps — if either is ever deleted (e.g. after a full `terraform destroy`, since Secrets live in the cluster, not in state), alerts or ArgoCD sync will silently stop working with no corresponding YAML to check for a fix.

Hardcoding inconsistency worth knowing about, not necessarily fixing: `helm/values/aws-load-balancer-controller.yaml.gotmpl` and `external-dns.yaml.gotmpl` both use `.gotmpl` + `requiredEnv` to inject the AWS account ID at apply time. `prometheus-stack.yaml` and `loki.yaml` could have used the same pattern but instead hardcode the literal account ID — and `manifests/fluentbit/fluent-bit.yaml` does too, despite not being one of the two files CLAUDE.md's ArgoCD section names as intentionally hardcoded. None of this is broken — the account ID doesn't change — but if asked "is this consistent," the honest answer is no, and now you know exactly where.

THE APPLICATION LAYER

The five microservices

None of these five container images were written or are owned by the team — they're pulled pre-built from AWS's public EKS Workshop reference app. The team's entire contribution is the platform running underneath them.

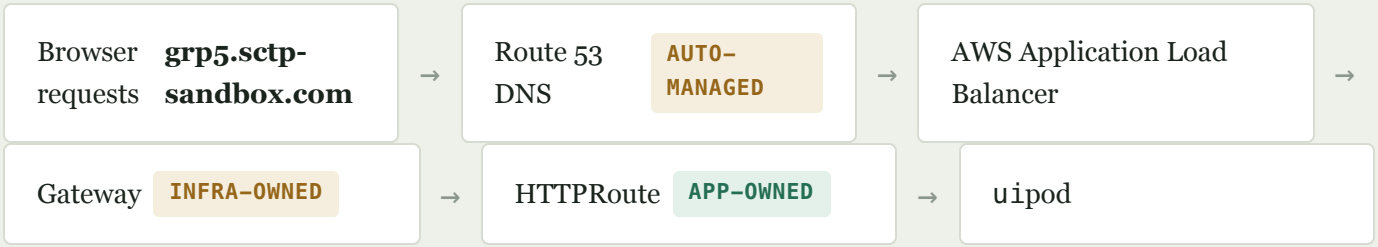
SERVICE	WHAT IT DOES	BUILT WITH	BACKED BY
ui	The storefront website — the only service a shopper's browser talks to directly. Calls the other four behind the scenes.	Java / Spring Boot	— (no database)

SERVICE	WHAT IT DOES	BUILT WITH	BACKED BY
`catalog`	The product list — what's for sale, prices, descriptions.	Go	MySQL 8.0 (StatefulSet)
`carts`	What's currently in a shopper's basket.	Java / Spring Boot	DynamoDB Local
`checkout`	Turns a cart into an order at the point of purchase.	Node.js	Redis 6.0
`orders`	Stores completed orders once checkout finishes.	Java / Spring Boot	PostgreSQL 16.1

NETWORKING

How a request gets in

The path a shopper's browser takes to reach the `ui` pod, and who's responsible for each hop.



Nobody manually creates the DNS record — **external-dns** notices the new route in the cluster and writes the Route 53 entry itself. Nobody manually clicks "create load balancer" in the AWS console either — the **AWS Load Balancer Controller** provisions and manages the ALB from the same Kubernetes resources.

Those resources are split deliberately in two: a **Gateway** (the "front door," owned by whoever runs the infrastructure) and an **HTTPRoute** (the "which path goes to which service" rule, owned by the application team). This is the newer Kubernetes networking standard — Gateway API — chosen over the older, more familiar `Ingress` resource specifically because that infra/app split mirrors how the rest of this repo is organised, and because it's the direction Kubernetes networking is actually heading.

KNOWING WHEN SOMETHING'S WRONG

Watching the system

Four separate pillars, all feeding the same Grafana dashboards, all running on the isolated `observability` node group described above — deliberately, so the system watching for a failure can't be taken down by the same failure it's supposed to catch.

- **Metrics**

A collector scrapes every pod for numbers — CPU, memory, request counts, error rates — every few seconds and feeds them into Prometheus. Grafana then queries Prometheus and draws the dashboards the team actually looks at.

- **Logs**

Fluent Bit runs on every server, tails every container's log output as it's written, and ships it off. Loki stores and indexes it, turning it into something searchable from inside Grafana — like a search engine, but for log lines.

- **Traces**

The trickiest of the three, because it means following one shopping request as it hops `ui → catalog → carts → checkout` to see exactly where time got lost. Normally that requires editing each service's source code — not an option here, since the team doesn't own that code.

Instead, the OpenTelemetry Operator injects tracing automatically at the moment a pod starts, based on an annotation on that pod — zero source code touched. Java and Node.js services get it via an extra init container; the Go service gets it through a lower-level technique (an eBPF sidecar) since Go doesn't support the same injection method. About 1 in 10 requests gets traced, to keep the volume manageable, and every trace ends up in AWS X-Ray, viewable straight from Grafana.

- **Alerts**

Prometheus continuously checks two rules against reality:

ALERT	SEVERITY	FIRES WHEN
`NodeNotReady`	critical	A server hasn't reported itself healthy for more than 15 seconds — usually because it was terminated or crashed.
`RetailStorePodsPending`	warning	One or more app pods have been stuck waiting to start for more than 5 seconds — the expected knock-on effect of losing a node.

When either fires, Alertmanager pushes a message straight into the team's Discord — a small trick makes this work: Discord happens to accept the exact webhook format Slack uses, so Alertmanager's built-in Slack integration works unmodified, just pointed at a Discord URL.

- **Cost**

Kubecost reads the same Prometheus metrics everything else uses and breaks the AWS bill down by Kubernetes namespace — so "how much does `checkout` cost us this month" gets an actual number, instead of one lump AWS invoice with no way to attribute it.

CONTINUOUS DELIVERY

How a change ships

After the very first manual apply, a human never runs a deploy command again — ArgoCD, itself deployed as an app in this repo, takes over watching the repo it lives in.

ArgoCD's `Application` resource is configured with two behaviours worth knowing by name, since they explain almost everything about how the cluster stays in sync:

- **selfHeal** — if anyone manually changes something inside the cluster by hand (say, via `kubectl edit`), ArgoCD notices the drift from what's in Git and quietly reverts it back

within a few minutes. Git — not whatever's currently running — is treated as the source of truth.

- **prune** — if a file is deleted from `manifests/` in the repo, the matching resource is deleted from the cluster too, automatically.

Because the repo is private, ArgoCD needs a read-only key to clone it — that's set up once, before the very first apply, so ArgoCD can start syncing immediately instead of sitting in an auth-error state.

PROVING IT, NOT JUST TRUSTING IT

Breaking it on purpose

Anyone can write an alert rule that looks correct on paper. The only way to know it actually fires under a real failure is to cause one — deliberately, safely, and on a schedule the team controls.

AWS Fault Injection Service (FIS) is aimed — via a narrowly scoped IAM role that can only terminate EC2 instances, nothing else — at exactly one target: 67% of the instances in the `application` node group. Never the `observability` node group, on purpose: that's the same isolation decision from the layers section above, and it's what stops this experiment from also killing the very system that's supposed to detect and report on the failure it causes.

1. FIS terminates roughly two-thirds of the application node group's real EC2 instances.
2. Those nodes stop reporting healthy — `NodeNotReady` starts counting down.
3. Because it's an EKS *managed* node group, AWS automatically launches replacement instances without anyone asking.
4. Kubernetes evicts the pods that were on the dead nodes and reschedules them onto whatever's healthy — which briefly leaves some pods `Pending`, tripping `RetailStorePodsPending`.
5. Both alerts fire, Discord gets two messages, and the Grafana panel turns red with "FIRING!"

6. Once the replacement nodes join and the pending pods land on them, both alerts clear themselves — no one touches anything.

THE REASONING

Why we chose this

Every major tool choice is written up as a full Architecture Decision Record in `docs/adr/`, including the options that were considered and rejected. Condensed to one line each:

ADR	CHOSE	OVER	BECAUSE
0001	Amazon EKS	Plain EC2 + Docker	Managed control plane, built-in self-healing/scaling, and the industry-standard way to demonstrate SRE practice.
0002	Gateway API	Classic Ingress	Where Kubernetes networking is actually heading, and it cleanly splits infra-owned routing from app-owned routing.
0003	Helmfile + Kustomize split	One tool for everything	Third-party software needs Helm's install/upgrade machinery; the team's own plain YAML doesn't need a chart at all.
0004	Self-hosted Prometheus/Grafana/Loki/OTel	CloudWatch, or paid SaaS	Free beyond storage already being paid for, and the skills transfer to real jobs better than a CloudWatch-only setup.
0005	Separate, tainted observability node group	One shared node group	So the chaos demo can kill app servers without also killing the system meant to detect the failure.
0006	S3 remote state with native locking	Local state, or S3 + DynamoDB	One shared AWS resource covers both storage and locking for five people applying from their own laptops.
0007	Kubecost	AWS Cost Explorer alone	Cost Explorer can't say "catalog cost $X this month" — only per-namespace breakdown can.

ADR	CHOSE	OVER	BECAUSE
0008	AWS FIS	In-cluster chaos tools, or manual CLI	Terminates real EC2 instances — an authentic test — without installing yet another in-cluster controller.
0009	ArgoCD	Manual `kubectl apply`, or Flux	Nobody has to remember to deploy, and its UI is far better for showing a live audience what's happening.
0010	OTel auto-instrumentation	Hand-editing each service's code	The team doesn't own the application source — auto-injection needs zero code changes.
0011	Discord (via Slack-compatible webhook)	Email, real Slack, PagerDuty	The team already lives in Discord, and it happens to accept Slack's webhook format for free.

HONEST ABOUT SCOPE

What we deliberately left out

Good defence in Q&A is naming these before someone else does. None of these are oversights — they're scope decisions for a one-month, five-person learning project, not a production system.

- **No HTTPS** — every service is plain HTTP; production would use TLS via ACM or cert-manager.
- **No CI pipeline** — no automated image building, testing, or scanning; the app images are pre-built and deployed as-is.
- **No secrets manager** — things like the Discord webhook URL sit in plain Kubernetes Secrets, not AWS Secrets Manager or Vault.
- **No multi-environment** — one cluster, one environment; production would separate dev/staging/prod with promotion gates.
- **No auto-scaling** — neither pods nor nodes scale automatically with load.
- **No RBAC** — every team member has cluster-admin access; production would scope permissions per role.

- **No network policies** — any service can talk to any other service inside the cluster, unrestricted.
- **Single region** — no cross-region redundancy or disaster recovery.

JARGON, DEFUSED

Glossary

The dozen terms most likely to come up in a question you weren't expecting.

ALB	AWS Application Load Balancer — the actual internet-facing thing that receives traffic, provisioned automatically by the load balancer controller.
CRD	Custom Resource Definition — teaches Kubernetes a brand-new type of object it didn't understand before. Nothing of that new type can be created until its CRD is installed.
GitOps	The practice of treating a Git repo as the single source of truth for what should be running, with a tool (here, ArgoCD) automatically reconciling reality to match it.
Helm chart	A pre-packaged, configurable install of a piece of software for Kubernetes — the unit Helmfile installs.
IaC	Infrastructure as Code — describing servers and networks as text files instead of manual console clicks, so the setup is repeatable and reviewable.
IRSA	IAM Roles for Service Accounts — lets a specific Kubernetes identity borrow scoped AWS permissions temporarily, with no access key or password stored anywhere.
Namespace	A named subdivision inside one cluster — each of the five services and each platform tool gets its own, keeping their resources separate.

Node group	A set of EC2 servers managed together as one unit inside the cluster. This project has two: `application` and `observability`.
PV / PVC	PersistentVolume / PersistentVolumeClaim — Kubernetes' way of attaching a real disk (here, an AWS EBS volume) to a pod so its data survives a restart.
Sampling	Tracing only a fraction of requests (10% here) instead of every single one, to keep the volume of trace data manageable and affordable.
Taint / toleration	A taint blocks anything from being scheduled onto a server unless it carries a matching toleration — used here to reserve the observability node group for monitoring pods only.
Webhook	A URL that a service calls automatically when something happens — used two different ways here: Kubernetes calls one to auto-inject tracing into new pods, and Alertmanager calls one to post into Discord.

Q&A PREP

Questions instructors will probably ask

Real gaps in this project, stated plainly, with the honest answer — pulled from the actual manifests and ADRs, not guessed. Click a question to reveal the answer and quiz yourself before you have to answer it live.

Architecture & trade-offs

Q **Isn't a full Kubernetes cluster overkill for five small microservices?**

Q **Why split deployment across Helmfile and Kustomize instead of one tool?**

Q **Why Gateway API instead of the far more common Ingress resource?**

Q **Why one NAT gateway instead of one per availability zone?**

Q **Gateway API + external-dns — what are the actual components, and how do they work together?**

Reliability & failure modes

Q **Every deployment shows `replicas: 1`. Doesn't the app actually go down during the node failure demo?**

Q **What happens to the data in carts, checkout, orders, or catalog's databases if their pod restarts?**

Q **You don't define any SLIs or SLOs. How do you know if the system is "reliable"?**

Q **`NodeNotReady` fires after only 15 seconds. Isn't that far too sensitive for a real production alert?**

Security

Q **Every team member has cluster-admin. What stops someone from accidentally deleting production?**

Q **The Discord webhook is stored as a plain Kubernetes Secret. Isn't that insecure?**

Q **How do you stop a compromised pod in one namespace from reaching another namespace's database?**

Q **Why is everything plain HTTP instead of HTTPS?**

Cost

- Q **What is this actually costing to run?**
- Q **Kubecost itself needs infrastructure to run — isn't a cost tool that costs money a bit circular?**

GitOps & deployment safety

- Q **ArgoCD auto-syncs with selfHeal and prune on. What stops a bad commit to main from immediately breaking the live app?**
- Q **How would you roll back a bad deploy, given nobody runs kubectl apply anymore?**
- Q **ArgoCD's repo-server can't run envsubst — why does that matter?**
- Q **Is ArgoCD set up as "app of apps," or one single Application?**

Chaos engineering rigor

- Q **Why terminate 67% of the application node group instead of 100%?**
- Q **Does this chaos test anything besides "an EC2 instance disappears"?**
- Q **Why does the experiment only ever target the application node group, never observability?**

Observability depth

- Q **Only 10% of requests are traced. How would you catch a bug that only shows up 1% of the time?**

Q **Metrics, logs, and traces all live on one dedicated node group. What if that node group itself goes down?**

Q **If someone tears the whole cluster down and rebuilds it, do the Grafana dashboards and historical metrics survive?**

Q **Why Fluent Bit for logs instead of Promtail or Grafana Alloy?**

Q **What are the different sources of logs, and where does each one go?**

Q **OpenTelemetry and Node Exporter both feed Prometheus. What's the actual difference in what they collect?**

Q **How are the app metrics actually collected, end to end?**

Q **Why use CloudWatch for traces?**

Q **How would you actually use traces for root-cause analysis?**

FOR THE ACTUAL PRESENTATION

Presenting this

Matches the running order in `PRESENTATION_OUTLINE.md` — who covers what, and which section above backs it up.

Arista

- Introduction & team, SRE/DevOps project overview
- Overall architecture overview — see The one-minute version and The three layers

Sze Kong	• Application architecture — see The five microservices • API Gateway architecture — see How a request gets in
Indy	• Observability architecture: metrics, logs, traces, EBS — see Watching the system • CD pipeline overview — see How a change ships
Gina	• Dashboards overview, alerts section walkthrough • Node failure demo (AWS FIS) — see Breaking it on purpose • Discord webhook alerts, DevOps Agent
Faizal	• ArgoCD demo (live replica-count change) — see How a change ships • Limitations & future improvements — see What we left out • Key takeaways
All	• Q&A — see Questions instructors will probably ask, plus Why we chose this and What we left out

- **Live URLs**

Store	`http://grp5.sctp-sandbox.com`
Grafana	`http://grp5-grafana.sctp-sandbox.com`
ArgoCD	`http://grp5-argocd.sctp-sandbox.com`
Kubecost	`http://grp5-kubecost.sctp-sandbox.com`
Prometheus	`http://grp5-prometheus.sctp-sandbox.com`

- **Two commands you'll need mid-demo**

```
# Grafana admin password
kubectl get secret -n monitoring prometheus-grafana -o jsonpath="
```

```
{.data.admin-password}" | base64 --decode

# ArgoCD admin password (username: admin)
kubectl get secret -n argocd argocd-initial-admin-secret -o jsonpath="
{.data.password}" | base64 --decode
```
